

“HỌC ĐỂ LÀM VIỆC”

C++ for Base

Contents



Inheritance



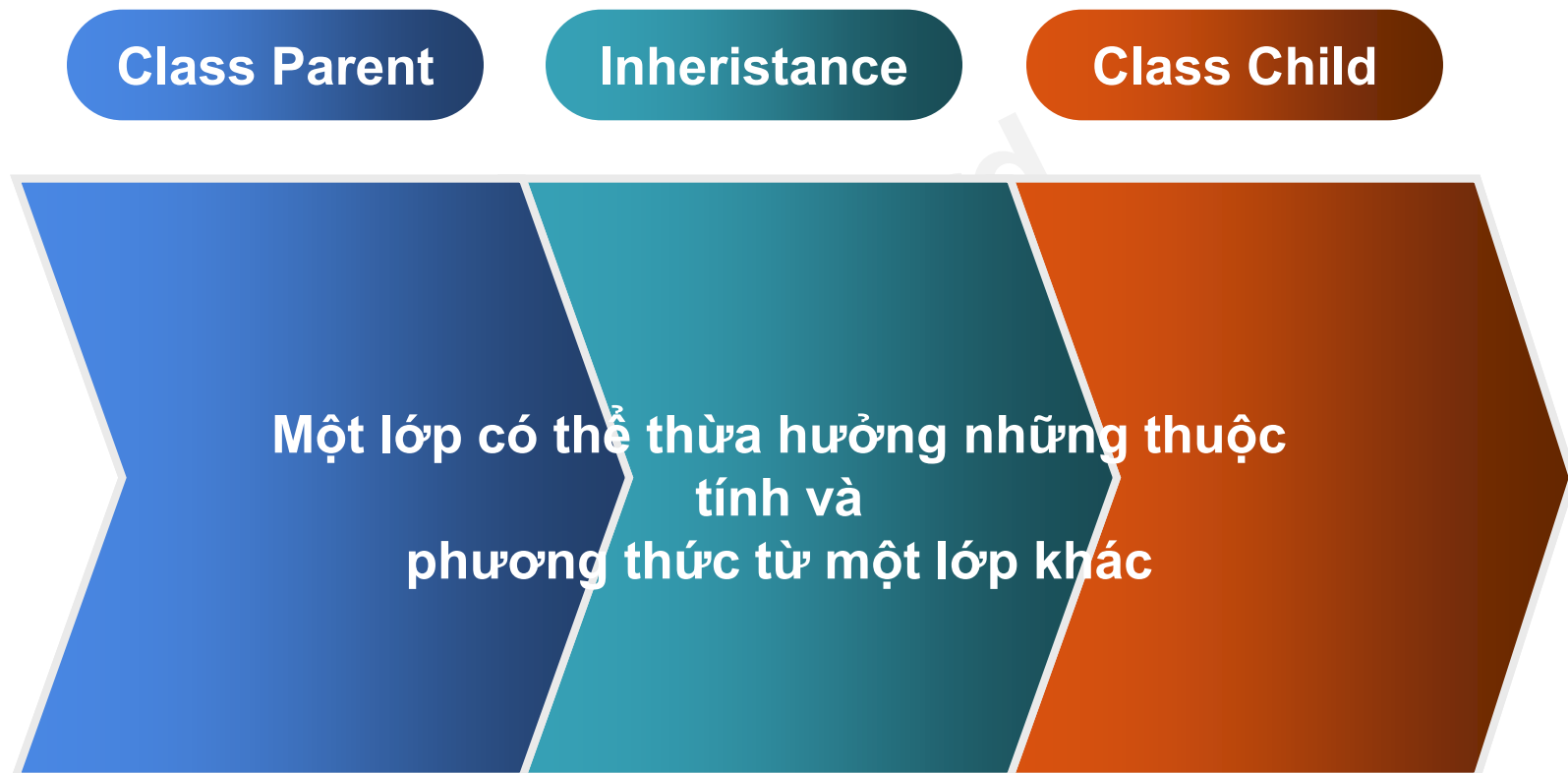
Polymorphism



Exercises

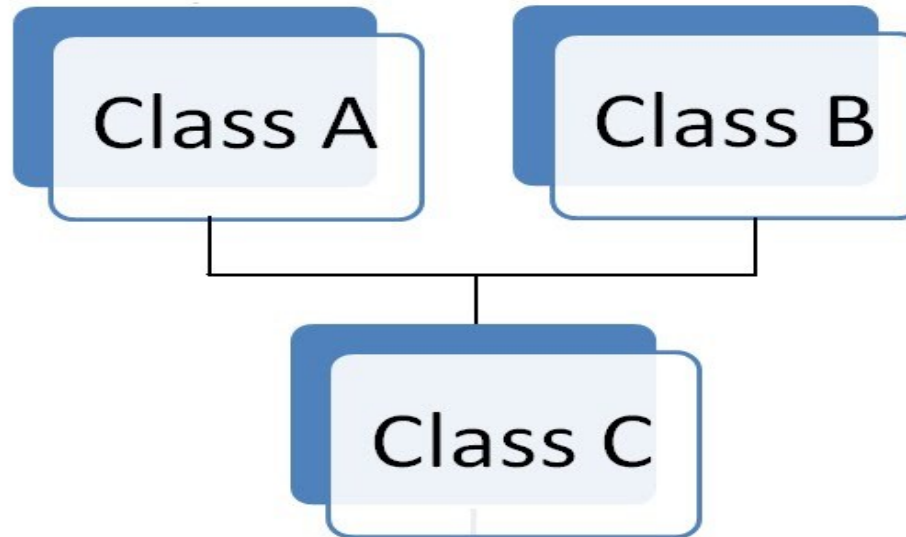
INHERITANCE

■ Inheritance of Objects Class



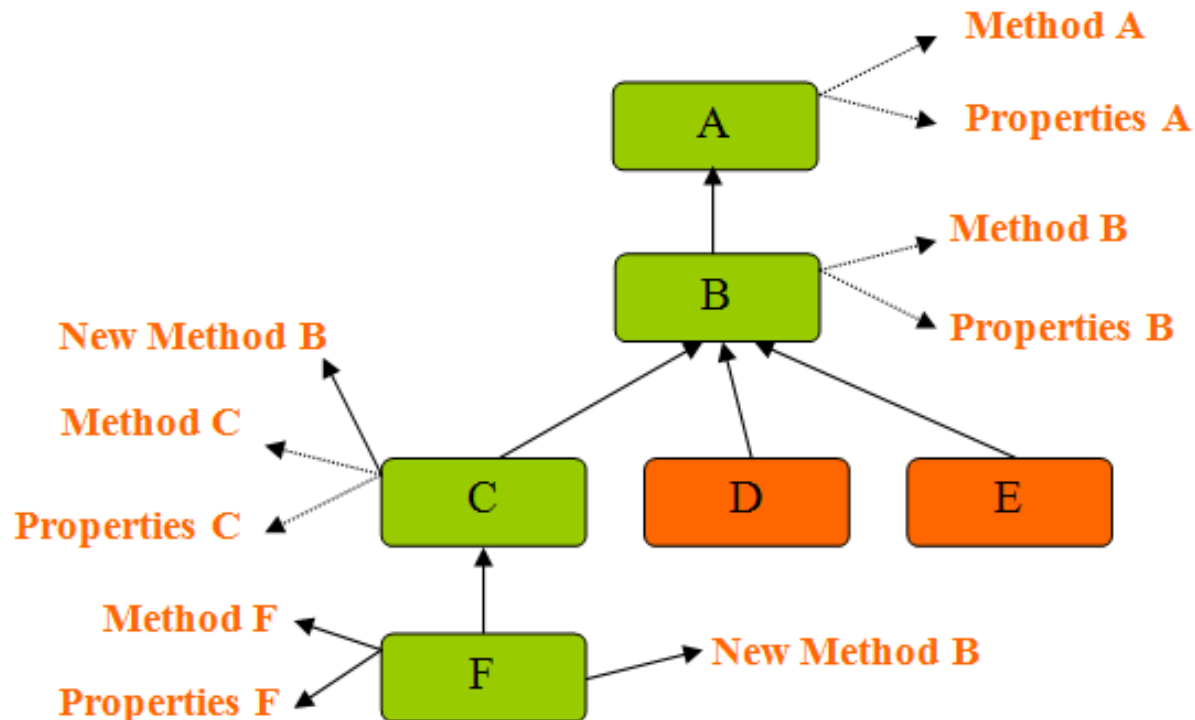
❖ Kế thừa (Inheritance)

- Một lớp được xây dựng thừa kế một lớp khác gọi là lớp dẫn xuất (lớp kế thừa). Lớp dùng để xây dựng lớp dẫn xuất gọi là lớp cơ sở



❖ Kế thừa (Inheritance)

- Một lớp được xây dựng thừa kế một lớp khác gọi là lớp dẫn xuất (lớp kế thừa). Lớp dùng để xây dựng lớp dẫn xuất gọi là lớp cơ sở



❖ Kế thừa (Inheritance)

- Sơ đồ trên chính là sự thừa kế nhiều mức. Trong đó có một số lưu ý như sau:
 - Một lớp có thể được dẫn xuất từ nhiều lớp cơ sở
 - Một lớp có thể dùng làm cơ sở cho nhiều lớp

❖ Kế thừa (Inheritance)

- Cú pháp khai báo

```
class Tên_lớp_kế_thừa : [Từ_khóa] Tên_lớp_cơ_sở  
{  
    //Viết nội dung xử lý của lớp kế thừa  
}
```

Nếu kế thừa từ nhiều lớp thì các lớp cách nhau bằng dấu phẩy “,”.

Từ khóa: private, public

❖ Kế thừa (Inheritance)

- Nếu không sử dụng từ khóa thì mặc định là **private** có nghĩa là tất cả các thành phần public của lớp cơ sở sẽ trở thành các thành phần private của lớp dẫn xuất.
- Kế thừa theo kiểu **public** thì tất cả các thành phần public của lớp cơ sở cũng là các thành phần public của lớp dẫn xuất.

❖ Kế thừa (Inheritance)

- Ví dụ:

```
class person: public animal
```

```
{  
  
}
```

```
class animal
```

```
{  
    private:  
        int somat;  
        int sochan;  
        char maulong[20];  
    public:  
  
}
```

❖ Kế thừa (Inheritance)

- **Lớp ảo:** Là lớp sử dụng từ khóa **virtual** trước khi khai báo tên lớp cơ sở để tránh sự nhập nhằng giữa các lớp kế thừa từ một lớp cơ sở.

Ví dụ:

```
class A
{
    public:
        int a;
}
```

```
class B: public A
{
    public:
        int b;
}

class C: public A
{
    public:
        int c;
}
```

```
class D: public B, public C
{
    public:
        int d;
}
```

❖ Kế thừa (Inheritance)

■ Lớp ảo

- Khai báo sử dụng lớp: D h;
- h.a = 1 //Lỗi

=> Chương trình C++ không thể nhận biết được thuộc tính a thừa kế thông qua B hay qua C và đưa ra thông báo lỗi Member is ambiguous: 'A::a' and 'A::a'

❖ Kế thừa (Inheritance)

■ Lớp ảo

Cú pháp sử dụng:

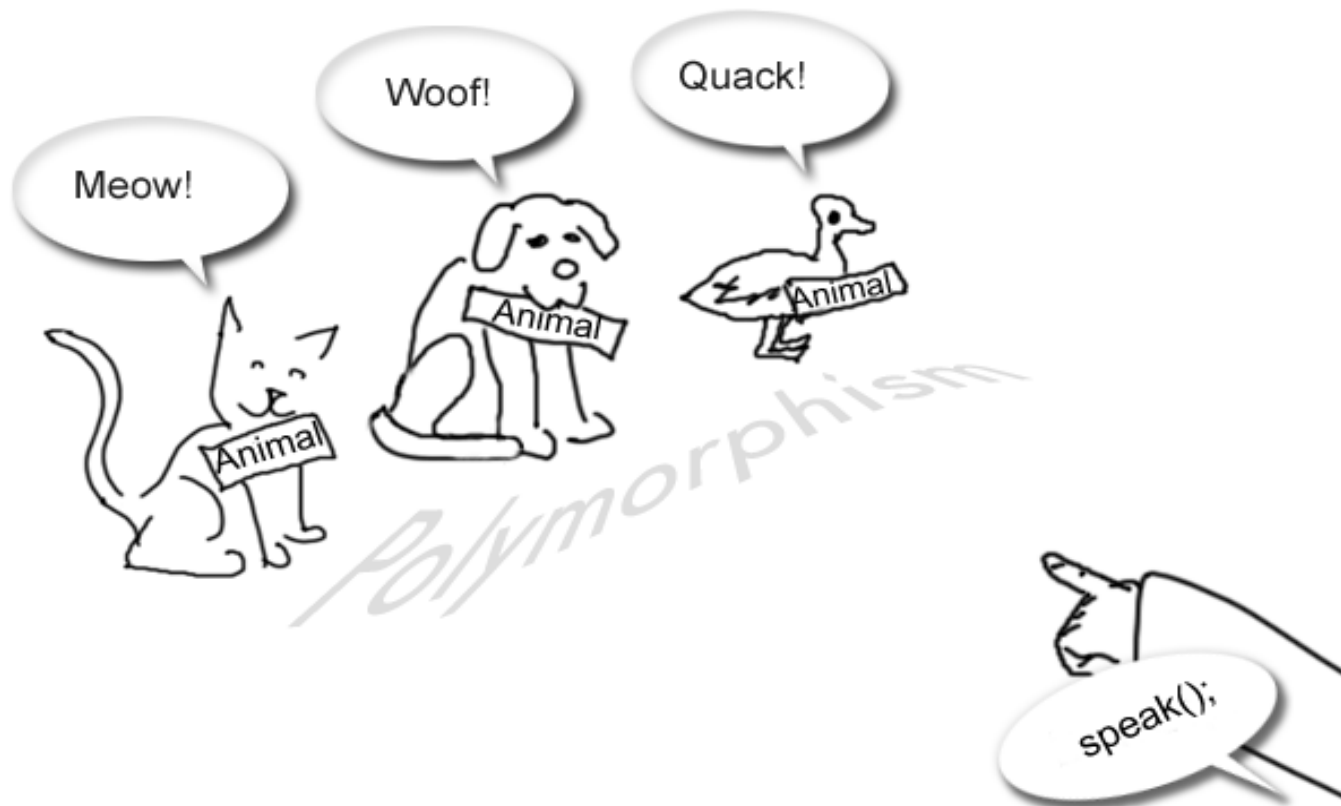
```
class C: virtual public A
{
    public:

        int c;
}
```

Các lớp cơ sở ảo (virtual) sẽ được kết hợp để tạo một lớp cơ sở duy nhất cho bất kỳ lớp nào dẫn xuất từ chúng.

POLYMORPHISM

❖ Đa hình (polymorphism) trong C++



❖ Đa hình (polymorphism) trong C++

- Là khả năng cho phép một hành vi (**method**) có thể được thể hiện dưới nhiều hình thức khác nhau mà không quan tâm tới chi tiết.
- Để tạo một method hỗ trợ tính đa hình khai báo với từ khoá virtual trước phương thức của lớp cơ sở sử dụng hoặc trước phương thức của tất cả các lớp.

Ví dụ: class people: virtual public animar hoặc

```
class animar{  
  
    public: virtual char* TiengKeu();  
  
}
```


❖ Đa hình (polymorphism) trong C++

■ Cú pháp khai báo Phương thức ảo:

- **virtual** [Kieu|void] Ten_Phuong_Thuc_Ao([Danh_sach_doi_so]);

Ví dụ: class animar{

public: virtual char* TiengKeu();

}

❖ Đa hình (polymorphism) trong C++

■ Lớp cơ sở trừu tượng:

- Một lớp cơ sở trừu tượng là một lớp chỉ được dùng làm cơ sở cho các lớp khác.
- Không có đối tượng nào của một lớp trừu tượng được tạo ra vì nó chỉ được dùng để định nghĩa một số khái niệm tổng quát, chung cho các lớp khác.
- Trong C++, thuật ngữ “**Lớp trừu tượng**” đặc biệt áp dụng cho các lớp có chứa các phương thức ảo thuần túy tức là phương thức mà nội dung của nó không có gì.

Exercises

“HỌC ĐỂ LÀM VIỆC”

Thank You !